

Unifying Streaming Data Reorganization and Interactive Real-Time Observability in Large Clusters

by
Ankush Jain

A DISSERTATION PROPOSAL
Submitted in partial fulfillment of
the requirements for the degree of

DOCTOR OF PHILOSOPHY
Electrical and Computer Engineering
Carnegie Mellon University

Committee:
Prof. George Amvrosiadis
Prof. Greg Ganger
Prof. Andy Pavlo
Prof. Vyas Sekar
Dr. Brad Settlemyer

CARNEGIE MELLON UNIVERSITY
Pittsburgh, Pennsylvania
July 2025

Contents

Contents	i
1 Introduction	2
1.1 Motivation and Scope	2
1.2 Status Quo: A Nascent In-Situ Landscape	2
1.3 Gaps and the Journey to Generality	2
1.4 Thesis Statement	3
1.5 Overview of Contributions	3
2 Background and Landscape	4
2.1 The BSP Paradigm and Synchronized Cluster Computing	4
2.2 Anatomy of Modern Clusters	4
2.3 Data Movement and I/O Challenges	4
2.4 In-Situ Analytics: Community Drivers and Status Quo	4
2.5 Scientific Code Ecosystem	5
3 CARP — Adaptive In-Situ Range Partitioning	6
3.1 Relevant Background and Architectural Rationale	6
3.2 Motivation and Context	6
3.3 Approach	6
3.4 Key Results	6
3.5 Broader Impact	6
4 AMR — Diagnosing and Optimizing Adaptive Mesh Codes	7
4.1 Background and Motivation	7
4.2 Load-Balancing Policies and Baselines	7
4.3 The Diagnostic Journey	7
4.4 Evolution of the Telemetry Pipeline	7
4.5 Key Results and Policy Development	7
4.6 Broader Lessons	8
5 ORCA — Generalizing Observability and Control	9
5.1 Background and Motivation	9
5.2 Approach and Architecture	9
5.3 Key Results and Capabilities	9
5.4 Remaining Work and Timeline	9
5.5 Broader Impact	9
References	10

Abstract

The Bulk Synchronous Parallel (BSP) paradigm powers some of our most critical computational tools, from large-scale scientific simulations to distributed machine learning. As these applications scale, their efficiency has datacenter-level impacts on both cost and time-to-discovery. Yet the tightly synchronized nature of BSP creates two distinct challenges: bursts of checkpointed state strain the data path, while global synchronization amplifies sensitivity to execution variability on the compute path. While traditionally addressed by separate communities and tools, this thesis demonstrates that both problems can be mapped to a common challenge: organizing and processing streaming data for insight.

We first address a data path challenge: efficiently enabling range queries on massive, streaming scientific datasets. Analytics on continuous physical attributes, such as energy bands, naturally map to range queries, but efficient querying requires expensive post-processing sorts. CARP is an adaptive range partitioner that reorders data in-situ during application I/O, adaptively tracking highly skewed and evolving distributions. As a result, CARP approximates the query performance of a sort with no ingestion overhead, achieving up to 5x faster performance than prior work.

We then turn to the compute phase, examining what is typically framed as a load-balancing problem in adaptive parallel codes. In practice, we found that genuine load imbalance was often masked by deeper sources of variability—hardware faults, system noise, and unpredictable interactions across the stack. Standard profiling tools fell short, forcing us to develop our own telemetry pipelines, which organically evolved into a query-driven analytics stack. This process led to CPLX, a tunable placement policy that balances compute load with communication locality, achieving up to 21.6% runtime improvement over optimized AMR baselines. More importantly, this work revealed that the central challenge in HPC performance is often not in solving imbalance, but in diagnosing what truly limits scalability.

Lessons from CARP’s adaptive control and the diagnostic imperative in AMR motivated the development of ORCA, a general-purpose framework for programmable, real-time analytics and control on streaming telemetry. ORCA synthesizes these insights by leveraging distributed query planning and a decoupled overlay architecture, built on Apache DataFusion, to efficiently reconstruct global system state from hierarchical aggregations. Its consistent control path—embodied by the Timestep-Linked Two-Phase Commit (TS2PC) protocol—enables robust, real-time application steering. Through declarative, query-driven dataflows, ORCA unifies observability and control, fundamentally transforming how we analyze, tune, and interact with large-scale computational systems.

1 Introduction

1.1 Motivation and Scope

The Bulk Synchronous Parallel (BSP) paradigm powers some of our most critical computational tools, from large-scale scientific simulations to distributed machine learning. These workloads—consisting of large parallel computations interspersed with I/O and global synchronization—currently use up to $O(10k)$ nodes [1–4] and continue to grow with time [5]. As these applications scale, their efficiency has datacenter-level impacts on both cost and time-to-discovery.

The tightly synchronized nature of BSP creates two unique challenges that strain modern computing infrastructure. On the data path, bursts of checkpointed state create massive I/O demands on the storage and analytics infrastructure [6]. On the compute path, global synchronization amplifies sensitivity to execution variability [7–9]. Performance bottlenecks that would be manageable in loosely coupled systems become critical scaling barriers when every worker must wait for the slowest participant.

1.2 Status Quo: A Nascent In-Situ Landscape

In-situ analytics emerged as a promising response to these BSP challenges, attempting to leverage idle computational resources during I/O phases to perform data analysis and organization tasks that would otherwise require expensive post-processing.

Early in-situ work focused on tightly coupled libraries such as ParaView Catalyst, though these were often limited to scripted visualization or basic filtering operations. Loosely coupled approaches like ADIOS and DataSpaces enabled more sophisticated co-processing workflows but introduced additional complexity and network dependencies.

Overlay networks including MRNet and LDMS provided scalable data reduction capabilities but offered limited programmability, finding primary use in monitoring rather than general analytics. In-situ indexing advanced with systems like DeltaFS implementing hash partitioning at scale, yet adaptive range partitioning remained an unsolved challenge.

Despite these advances, most in-situ systems remained point solutions—valuable within their specific domains but narrow in scope, with limited programmability or generality across different use cases.

1.3 Gaps and the Journey to Generality

On the data path, the available toolkit for in-situ analytics remained fundamentally incomplete. Most efforts focused on specific tasks such as visualization or compression, leaving open critical challenges including adaptive range partitioning of streaming data.

On the compute path, the prevailing struggle centered on diagnosing and fixing performance issues. While widely understood as fundamentally a load balancing problem, practitioners were

plagued by noisy telemetry and insufficient diagnostic tools to distinguish workload imbalance from environmental factors.

This thesis initially approached these as two separate research problems. Only through extensive empirical systems work did it become clear that both represented instances of a deeper, more general challenge: organizing and processing streaming data effectively for both insight generation and real-time control.

1.4 Thesis Statement

Streaming, programmable analytics and control can unify and solve both data and compute challenges in synchronized cluster computing.

1.5 Overview of Contributions

This thesis presents three major contributions that progressively develop this unified approach:

CARP fills the last major gap in in-situ data organization by providing adaptive range partitioning of streaming data, completing the toolkit of fundamental partitioning primitives for in-situ analytics.

CPLX/AMR develops programmable, empirical workflows and tunable placement policies for real-world performance diagnosis and optimization, demonstrating the necessity of full-stack telemetry over traditional approaches.

ORCA generalizes these insights into a unified, real-time framework for observability and control at cluster scale, providing a programmable substrate for both data analytics and system optimization.

2 Background and Landscape

2.1 The BSP Paradigm and Synchronized Cluster Computing

- What is BSP? Why is it foundational for both HPC and large-scale ML?
- Timestepped execution, collectives, and synchronization barriers.
- Impact on application design: sensitivity to stragglers and the importance of uniform progress.

2.2 Anatomy of Modern Clusters

- Cluster resources: nodes, memory hierarchies, accelerators (e.g., GPUs).
- **Networking:**
 - HPC: specialized high-speed fabrics (e.g., InfiniBand, Aries) for collectives, plus Ethernet control networks.
 - ML clusters: backend networks for data, frontend for orchestration.
- Storage infrastructure: parallel filesystems (Lustre, GPFS, BeeGFS), burst buffers, SSD tiers.

2.3 Data Movement and I/O Challenges

- Bursty checkpointing and collective I/O phases in scientific apps.
- The network/storage bandwidth gap as a persistent architectural problem.
- Checkpointing and trace collection as stressors for both storage and system performance.
- The difference between data written for analysis (HPC) and for recoverability (ML).

2.4 In-Situ Analytics: Community Drivers and Status Quo

- Traditional “persist-then-analyze” model and its breakdown at scale.
- Why in-situ emerged: reducing time-to-insight, bandwidth needs, and storage costs.
- Overview of major in-situ paradigms:
 - Tightly coupled: libraries embedded in simulation.
 - Loosely coupled: co-processing with staging/burst nodes.
 - Overlay networks: aggregation and scalable monitoring.
 - In-situ indexing: hash-based (DeltaFS), range (missing).
- Limitations of each and the context for new approaches.

2.5 Scientific Code Ecosystem

- Landscape of open-source and institutional codes: how science gets done at scale.
- **VPIC:** what it is, scientific significance, I/O demands.
- **AMR codes:** (e.g., Parthenon, FLASH, Enzo) for multiscale, adaptive modeling.
- Other representative codes and domains (brief nod to climate, molecular dynamics, etc.).
- Common challenges: data volumes, run duration, complexity, and the resulting analysis needs.

3 CARP — Adaptive In-Situ Range Partitioning

3.1 Relevant Background and Architectural Rationale

- Prior in-situ approaches were either tightly coupled to the application (hard to adapt and extend) or focused on fast, unordered ingestion (hash partitioning), sacrificing query efficiency.
- Range queries are critical in scientific analysis, but traditional solutions required expensive post-processing sorts—architecturally at odds with the streaming, in-memory nature of modern HPC I/O.
- The network-to-storage bandwidth gap and the bursty, append-heavy I/O patterns of checkpointing make streaming, adaptive, one-pass reorganization architecturally optimal.

3.2 Motivation and Context

- Adaptive range partitioning was the last missing primitive for general in-situ data organization.
- Scientific datasets are highly skewed and dynamic, demanding solutions that can adjust in real time.

3.3 Approach

- CARP adaptively reorganizes data during application I/O, discovering and adjusting range partitions on the fly.
- Operates fully in-situ, without extra passes or reordering after the fact.

3.4 Key Results

- Delivers near-sorted query performance for range queries, with zero ingestion overhead.
- Outperforms prior work: up to 5x faster than post-processing and up to 100x lower query latency than in-situ indexing baselines.
- Robust across extremes of skew and evolving data distributions.

3.5 Broader Impact

- Completes the set of architectural primitives for in-situ data organization.
- Introduces the “state-driven adaptive control loop” pattern, a theme carried forward in the thesis.

4 AMR — Diagnosing and Optimizing Adaptive Mesh Codes

4.1 Background and Motivation

- Block-structured AMR (Adaptive Mesh Refinement) codes are widely used for large-scale simulations in physics and engineering, but are notoriously difficult to optimize at scale.
- Synchronization and frequent mesh refinement create straggler effects; performance is traditionally diagnosed as a load-balancing problem.

4.2 Load-Balancing Policies and Baselines

- Common practice: work placement policies (e.g., Space-Filling Curves, naive block distribution) assume uniform costs and static assignment.
- Prior literature and production codes rely on heuristics; empirical tuning is difficult, and “obvious” improvements often fail at scale.

4.3 The Diagnostic Journey

- Initial attempts to optimize placement exposed more complex bottlenecks: hardware faults, system noise, and non-obvious sources of variability.
- Standard profiling tools were insufficient—transient effects and cross-stack anomalies went undetected.

4.4 Evolution of the Telemetry Pipeline

- Necessity drove a bottom-up diagnostic process: custom code instrumentation, direct CSV logging, and iterative data analysis.
- The ad-hoc telemetry stack evolved into a query-driven, structured workflow, borrowing concepts from modern OLAP and observability.
- Adoption of programmable tools (eBPF, OLAP engines) enabled the root-cause diagnosis that classical HPC tools could not deliver.

4.5 Key Results and Policy Development

- The CPLX placement policy: a tunable hybrid balancing load and communication locality, outperforming prior baselines by up to 21.6%.
- Main finding: true bottlenecks often lie outside what classical load balancing or placement can solve—highlighting the need for deeper, programmable, telemetry-driven diagnosis.

4.6 Broader Lessons

- Performance optimization for modern AMR codes is fundamentally empirical and diagnostic, not purely algorithmic.
- Motivates the need for a general, programmable approach to cluster telemetry and control—setting the stage for ORCA.

5 ORCA — Generalizing Observability and Control

5.1 Background and Motivation

- Lessons from CARP and AMR revealed the need for a general, programmable framework for both analytics and real-time control in large clusters.
- Existing tools provided either narrow observability, limited programmability, or ad-hoc steering—none delivered unified, declarative, scalable control.

5.2 Approach and Architecture

- ORCA introduces a decoupled overlay network: application, aggregators, and controller communicate via high-performance RPC.
- Built on distributed query planning (Apache DataFusion) for SQL-driven, in-memory analytics on streaming telemetry.
- Real-time control via a Timestep-Linked Two-Phase Commit (TS2PC) protocol ensures consistent, synchronized updates across all ranks.

5.3 Key Results and Capabilities

- Demonstrates scalable, low-overhead analytics at line-rate bandwidths, enabling “analyze-then-persist” at cluster scale.
- Makes dynamic, cluster-wide control practical—enabling online tuning, interactive workflows, and safe application reconfiguration in situ.
- Provides a declarative, extensible substrate for both scientific introspection and systems-level observability.

5.4 Remaining Work and Timeline

- TODO

5.5 Broader Impact

- ORCA subsumes fragmented observability and steering solutions under a unified, data-centric paradigm.
- Establishes the general, programmable control loop as a foundational primitive for future HPC and ML systems.

References

- [1] Sambit Das et al. “Large-Scale Materials Modeling at Quantum Accuracy: Ab Initio Simulations of Quasicrystals and Interacting Extended Defects in Metallic Alloys”. In: SC ’23: International Conference for High Performance Computing, Networking, Storage and Analysis. 2023.
- [2] Ryan Stocks et al. “Breaking the Million-Electron and 1 EFLOP/s Barriers: Biomolecular-Scale Ab Initio Molecular Dynamics Using MP2 Potentials”. In: SC ’24: International Conference for High Performance Computing, Networking, Storage and Analysis. 2024.
- [3] Aaron Grattafiori et al. *The Llama 3 Herd of Models*. 2024. arXiv: 2407.21783 [cs]. Pre-published.
- [4] *Meta Trains Llama 4 on a 100,000+ H100 GPU Supercluster - NADDOD Blog*.
- [5] Robi Rahman. *AI Training Cluster Sizes Increased by More than 20x since 2016*. Epoch AI. 2024.
- [6] Assaf Eisenman et al. “{Check-N-Run}: A Checkpointing System for Training Deep Learning Recommendation Models”. In: 19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22). 2022.
- [7] Torsten Hoefer, Timo Schneider, and Andrew Lumsdaine. “Characterizing the Influence of System Noise on Large-Scale Applications by Simulation”. In: SC ’10: The 2010 ACM/IEEE International Conference for High Performance Computing, Networking, Storage and Analysis. 2010.
- [8] Fabrizio Petrini, Darren J. Kerbyson, and Scott Pakin. “The Case of the Missing Supercomputer Performance: Achieving Optimal Performance on the 8,192 Processors of ASCI Q”. In: SC ’03: International Conference for High Performance Computing, Networking, Storage and Analysis. 2003.
- [9] Orcun Yildiz et al. “On the Root Causes of Cross-Application I/O Interference in HPC Storage Systems”. In: 2016 IEEE International Parallel and Distributed Processing Symposium (IPDPS). 2016.